保险系统测试实战手册 概念、真实问题与提速

测试最耗时间的通常不是“点按钮”，而是三件事：需求不清导致返工，测试数据和环境难准备，失败用例要花大量时间定位是真bug还是脚本或环境问题。大部分提速手段都围绕这三点展开。

第一部分讲清楚保单、Selenium、Appium 和所需技能；第二部分按“需求 → 用例 → 数据环境 → 执行 → 缺陷定位 → 回归 → 自动化维护 → UAT上线”八个阶段，列出实际工作中常见的具体问题、原因和优化办法。问题来自保险、金融类系统测试的普遍做法，属于行业通用经验，不是富卫内部资料。

一、基础概念

1. 保单是什么

业务基础

保单就是保险合同：客户交保费，保险公司承诺在约定的风险发生时赔钱。对测试来说，保单是系统里最核心的一条数据，几乎所有功能都围绕它的创建、变更和赔付展开。

要素	含义	测试时常关注的点
投保人	买保险、交钱的人	年龄、证件、与被保险人关系是否合法
被保险人	受保障的人	投保年龄上下限、职业类别、健康告知
受益人	出险后拿钱的人	多受益人比例相加是否等于100%
保额	最多赔多少	最低/最高保额、超额是否触发人工核保或体检
保费	每期交多少钱	按年龄、性别、缴费期、是否吸烟算出的金额是否与费率表一致
保险期间 / 缴费期	保多久 / 交多少年	到期日计算、闰年、月末日期
犹豫期 / 等待期	可无损退保的期限 / 生效后暂不赔付的期限	犹豫期内退保全额退款，等待期内出险不赔

保单生命周期：投保 → 核保（判断能不能保、要不要加费）→ 承保出单 → 续期交费 → 保全（改地址、加减保额、改受益人等）→ 理赔 → 满期或退保。测试要覆盖的就是这条链上每个环节，以及环节之间的数据是否一致。

2. Selenium 是什么

网页自动化

Selenium 是一个开源工具，让程序代替人去操作浏览器：打开网页、点击、输入、读取页面内容。它通过 WebDriver 协议（W3C 标准）驱动 Chrome、Edge、Firefox、Safari 等浏览器。

- 典型用途：自动走完网页端投保流程、后台核保系统操作、代理人门户的回归测试。
- 核心组成：WebDriver（控制单个浏览器）、Selenium Grid（多台机器、多浏览器并行执行）、Selenium IDE（录制回放，适合入门）。
- 支持语言：Python、Java、C#、JavaScript 等；富卫广州 JD 要求的是 Python。
- 局限：只能操作浏览器，不能直接测 App；页面频繁改版时脚本容易失效。

一个最小例子，意思是“打开投保页，输入年龄，点下一步”：

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("https://test-env.example.com/quote")
driver.find_element(By.ID, "age").send_keys("30")
driver.find_element(By.CSS_SELECTOR, "button.next").click()
assert "保费" in driver.page_source
```

3. Appium 是什么

APP 自动化

Appium 是手机端的“Selenium”：同样基于 WebDriver 协议，用代码操作 Android 和 iOS 上的 App。同一套写法可以测原生 App、混合 App（App 里嵌网页）和手机浏览器。

- Android 用 UiAutomator2 驱动，iOS 用 XCUITest 驱动；Appium 2 起驱动按需单独安装。
- 测 iOS 必须有 Mac 电脑和 Xcode；测 Android 需要 Android SDK 和 adb。
- 用 Appium Inspector 查看 App 页面元素，找到按钮的 id、文字或位置。
- 典型用途：客户 App 的投保、拍照上传理赔材料、人脸识别前后的流程、推送消息跳转。
- 难点：真机和模拟器差异、不同手机品牌系统弹窗不同、App 内嵌网页需要切换上下文。

4. 需要什么样的技术

技能清单

层次	具体技能	用在哪里
测试基础	等价类、边界值、判定表、场景法、正交法	设计测试用例
编程	Python：函数、类、异常处理、读写 Excel/JSON/YAML	写自动化脚本、造测试数据
测试框架	pytest（用例组织、fixture、参数化）、Allure（测试报告）	管理和运行脚本
前端基础	HTML 结构、CSS 选择器、XPath、浏览器开发者工具	定位页面元素、看接口请求
接口	HTTP、JSON、Postman、requests 库、鉴权（Token）	接口测试、绕过界面直接造单
数据库	SQL 查询、关联表、事务	核对保单数据是否正确落库
工程化	Git、Jenkins / GitLab CI / Azure DevOps、Docker 基础、Linux 常用命令、看日志	脚本接入流水线、排查环境问题
移动端	adb 命令、Appium Inspector、模拟器与真机调试	App 自动化
性能	JMeter、并发与响应时间概念	压测
协作工具	Jira（缺陷与需求）、Confluence（文档）、TestRail / Zephyr / Xray（用例管理）	日常流程
业务知识	保单生命周期、核保规则、费率、保全项目	判断结果对不对
语言	英文读写（JD 明确要求）	与港澳、东南亚市场同事沟通、读需求文档

二、各阶段的真实问题与提速办法

阶段1：需求评审

返工源头

需求阶段漏掉的问题，到测试阶段才发现，修复成本通常最高。

具体问题	真实场景	优化办法
验收标准模糊	User Story 只写“支持在线修改受益人”，没说能否改成非直系亲属、改完是否需要重新签名	评审时用“给定/当/那么”（Given/When/Then）格式逐条补齐验收标准，未写明的不进开发
边界规则缺失	“投保年龄 18–60 岁”，没说按周岁还是保险年龄，生日当天算不算	测试提前列出边界问题清单，让 BA 书面确认
多市场规则差异	同一产品在香港和泰国的犹豫期、币种、证件类型不同，需求只写了一个市场	需求模板加“适用市场”和“市场差异”字段
需求中途变更	测试到一半，业务把费率表换了一版，已执行用例全部作废	需求冻结点 + 变更必须登记 Jira 并评估测试影响
上下游影响没人提	改了核保规则，没人想到会影响已出单保单的续保	评审时固定问一句“哪些下游系统会受影响”，画出系统调用图
测试参与太晚	开发已经写完才通知测试看需求	测试参加需求澄清会，开发前完成用例初稿（测试左移）

阶段2：写测试用例

覆盖与效率

具体问题	真实场景	优化办法
组合爆炸	保费受年龄（18–60）× 性别 × 缴费期（5/10/20年）× 是否吸烟 × 保额档影响，全组合上万条	用正交法或成对组合（Pairwise）工具压缩到几十条；边界年龄单独补测
预期结果写不出来	用例写“保费计算正确”，但不知道正确值是多少	向精算拿费率表，用 Excel 或脚本按公式算出期望值，作为测试数据源
只写正常流程	没覆盖支付超时、重复点击提交、中途退出再回来	每个功能固定补“异常清单”：网络中断、重复提交、并发、超时、回退、会话过期
用例粒度不统一	有人一条用例 30 步，有人一条只测一个字段	团队统一模板：一个用例一个验证目的，步骤不超过约 10 步
用例与需求对不上	上线前问“这个需求测全了吗”，答不上来	在用例管理工具里建需求追溯矩阵，每条需求关联用例
老用例失效没人删	用例库几千条，三分之一是旧版本功能	每个迭代结束清理一次，标记“废弃”而不是直接删除
写用例本身耗时	每个迭代花 2–3 天手写用例	复用用例模板库；用 AI 根据需求生成初稿，测试人工审核补充

阶段3：测试数据与环境

最常见卡点

保险测试的数据特别难造：要测“理赔”，得先有一张已生效、过了等待期的保单。

具体问题	真实场景	优化办法
造一张可用保单要很久	手工走投保 → 核保 → 支付 → 出单，一张要 15–30 分钟	写造数脚本，直接调接口出单，一张降到几秒
时间相关场景测不了	要测“保单满一年续期”“过了 90 天等待期理赔”，不可能真的等	测试环境支持修改系统日期（时间穿越），或直接改库里的生效日期
数据被别人用掉	准备好的测试保单被同事拿去做退保，自己的用例跑不下去	按人或按用例隔离数据；每条自动化用例自己造数、自己清理
不能用生产数据	客户身份证、电话、病历属于敏感个人信息	用脱敏数据或生成假数据（如 Faker 库），证件号按校验规则生成
第三方依赖不可用	支付网关、银行扣款、短信、公安身份核验在测试环境不稳定或要收费	用 Mock 服务模拟返回成功、失败、超时
环境被频繁重新部署	测到一半，别人往同一环境发了新版本	环境预约表；按团队分环境；部署前群里通知
环境配置和生产不一致	测试环境开关打开了，生产没开，上线后功能不生效	配置纳入版本管理，上线前做配置比对
多市场多时区	香港、泰国、印尼时区和节假日不同，批处理跑批时间不同	用例里明确时区，跑批场景在对应市场时间验证

阶段4：测试执行

具体问题	真实场景	优化办法
提测版本质量太差	提测后登录都失败，测试一上午什么都测不了	设冒烟测试准入门槛，冒烟不过直接打回开发
界面通过、数据错误	页面显示投保成功，数据库里受益人比例存成了 0	关键步骤后加数据库或接口校验，不只看页面
只测了一端	网页端改了地址，App 端显示的还是旧地址（缓存）	多端一致性用例：一端改、另一端查
兼容性覆盖不全	某品牌安卓手机上，上传身份证照片后 App 闪退	按用户占比选核心机型；用云真机平台扩大覆盖
多语言显示	泰文、印尼文比英文长，按钮文字被截断	每种语言做一次界面截图检查，可用截图比对工具自动化
进度不透明	经理问“还剩多少”，只能估一个百分比	用例管理工具实时统计执行率、通过率、阻塞数
被阻塞干等	一个缺陷挡住后面 20 条用例	阻塞缺陷优先级最高，当天同步；先测不依赖它的模块

阶段5：缺陷提交与 Badcase 定位

最考验能力

Badcase 指结果不符合预期的具体案例。定位的目标是回答三个问题：问题出在哪一层、能否稳定复现、是谁的问题。

标准定位路径：看页面现象 → 打开浏览器开发者工具看接口请求和返回 → 判断是前端没传对还是后端返回错 → 查后端日志（按订单号或请求 ID 搜）→ 查数据库确认数据 → 查是否和配置、第三方、环境有关 → 找出最小复现步骤。

具体问题	真实场景	优化办法
缺陷描述不清	只写“保费算错了”，开发无法复现，来回问三轮	缺陷模板必填：环境、版本、账号、保单号、步骤、预期、实际、截图、接口请求和返回、日志片段
偶现问题难复现	十次里有一次支付后保单状态卡在“待出单”	记录发生时间和请求 ID 查日志；怀疑并发或异步消息丢失，用脚本循环 100 次复现
前后端互相推	前端说后端返回错，后端说前端传错	抓接口请求作为证据，直接定位到哪一方
“这不是 bug，是需求”	开发认为按需求就是这样做的	拿需求文档原文对照；需求本身有歧义时拉 BA 三方确认
精度问题	保费 1234.565 页面显示 1234.57，数据库存 1234.56	确认四舍五入规则和在哪一层取整，写进需求
时区与日期	香港时间晚上 11 点投保，保单生效日显示成第二天	检查服务器、数据库、前端各用什么时区
严重程度争议	测试定为严重，开发认为一般	团队统一分级标准，例如“影响出单或金额 = 严重”
同类问题反复出现	每个新产品上线都有费率表导入错误	做缺陷根因分析，按类型统计，针对高频类型加专项检查或自动化校验

阶段6：回归测试

具体问题	真实场景	优化办法
全量回归跑不完	回归用例 2000 条，手工要一周，但迭代只有两周	用例分级：P0 冒烟、P1 核心、P2 全量；每次发版只跑 P0+P1
不知道改动影响哪里	开发只说“改了一个公共方法”	要求开发提供改动清单；用代码覆盖率或调用链工具找出受影响功能
修一个 bug 引入新 bug	修了保全改地址，导致理赔通知寄错地址	每个缺陷修复后，跑与它相关的模块回归，不只验证那一个点
回归总在最后一天挤	开发拖延提测，回归时间被压缩	自动化回归每晚跑，问题提前暴露，不等到最后
串行执行太慢	500 条 UI 自动化串行要 6 小时	Selenium Grid 或云平台并行执行，按模块拆分，降到 1 小时内

阶段7：自动化脚本开发与维护

长期成本

自动化的真正成本在维护，不在编写。页面一改版，几十条脚本同时报错是常态。

具体问题	真实场景	优化办法
元素定位不稳定	XPath 写成 /div[3]/div[2]/button，前端加一个 div 就全失效	与前端约定给关键元素加固定的 data-testid 属性；用 Page Object 模式集中管理定位
Flaky 用例（时过时不过）	同一脚本今天过、明天失败，页面加载慢时点击了还没出现的按钮	不用固定 sleep，改用显式等待（等元素可点击）；失败自动重试一次并单独标记
测试数据写死	脚本里写死保单号，那张保单被退保后脚本永远失败	每次运行前自动造数，数据与脚本分离
App 系统弹窗干扰	安卓权限弹窗、iOS 通知弹窗打断流程	Appium 启动参数自动授予权限；封装通用弹窗处理
混合页面切换	App 内嵌的投保网页找不到元素	在 Appium 中切换到 WebView 上下文再定位
验证码与人脸识别	登录短信验证码、活体检测无法自动化	测试环境配置万能验证码或白名单账号，跳过活体检测
UI 自动化太多太慢	所有检查都放在 UI 层	把业务规则校验下沉到接口层，UI 只保留少量主流程
报告看不懂	失败只显示“元素未找到”	用 Allure 报告，失败时自动截图、录屏、附带接口日志
没人看失败结果	每晚自动跑，失败了一周没人处理	失败结果自动推送到 Teams 或企业微信，指定值班人当天处理
投入产出算不清	领导问“自动化省了多少时间”	统计自动化覆盖率、每次节省的人工工时、发现的缺陷数

阶段8：UAT、上线与上线后

具体问题	真实场景	优化办法
业务方没时间验收	核保、理赔同事日常工作忙，UAT 一拖再拖	提前约好 UAT 时间；测试准备好数据和操作说明，业务方只需确认结果
UAT 才发现需求理解错	业务说“我要的不是这个”	开发中期做一次演示，提前让业务看
跨时区沟通慢	广州团队提问，泰国业务第二天才回复	每日固定同步时间；问题集中列清单一次问完
上线窗口紧	只能在周末夜间停机上线，验证时间只有 1–2 小时	上线后冒烟用例全部自动化，几分钟内给出结论
生产不能随便下单	生产环境验证投保不能产生真实保单	使用专门的生产测试账号，或验证到支付前一步；产生的测试单按流程撤销
回滚标准不清	上线后发现问题，不确定要不要回滚	上线前写好回滚条件，例如“核心出单失败即回滚”
线上问题回流	客户投诉某场景出错，测试环境复现不了	拿生产日志和脱敏数据复现，修复后把该场景补进回归用例

三、提速手段汇总：先做哪些

按投入与收益排序

优先级	手段	解决哪类耗时	投入
1	造数脚本（接口直接出单）	准备测试数据	低
2	冒烟测试自动化 + 提测准入	版本质量差导致的空等	低
3	缺陷模板 + 接口抓包作证据	缺陷来回沟通	低
4	测试左移：开发前写好验收标准和用例	需求返工	中
5	第三方 Mock 服务	环境不稳定、外部依赖	中
6	接口自动化覆盖业务规则	回归时间长	中
7	回归用例分级 + 每晚定时执行	回归挤在最后	中

优先级	手段	解决哪类耗时	投入
8	并行执行（Grid / 云真机）	脚本执行慢	中高
9	稳定定位属性 + Page Object	脚本维护	中，需前端配合
10	AI 辅助生成用例、分析失败日志	写用例、定位	中，需人工审核

Made with Dia